Hisaab

Frontend guide: architecture and every feature

What was built in step 1 (the UI), how the pieces fit together, how REST and GraphQL share the work, what every screen does, and the rules the backend will need to follow.

Prepared for Divas Sharma · 5 November 2026

Frontend in `Hisaab/frontend`

Contents

1 Start here: run the app
2 The big picture
3 Folder and file map
4 How data moves: REST and GraphQL
5 Screens, one by one
6 The rules (money, splits, balances)
7 Who can do what
8 Draft API for step 2 (REST + GraphQL)
9 Draft database tables for step 2
10 Design system and UX reasons
11 Connecting the real backend (step 5)
12 What was tested, and limits

How to read this

Read sections 1 to 5 first. They explain what exists and how it works. Sections 6 to 9 are the part that matters for your backend: the rules, the permissions, and a first draft of the endpoints and tables. That draft is a starting point for step 2 (API contract). You decide the final version.

The project in one paragraph

Hisaab is an expense splitter for hostel rooms, flats and trips. Friends make a group. Anyone in the group adds an expense, for example "I paid ₹1,200 for dinner, split between 4". Hisaab works out who owes whom. It then suggests the fewest payments that clear the group. On a phone, one tap opens a UPI app with the payee and the exact amount already filled in.

Where step 1 stands

- Every v1 screen is built and works in the browser.
- The app runs on **mock data**: a small fake database saved in the browser. There is no backend yet.
- Every function the screens call is written in the shape of a real API call. Each one is labelled with the endpoint it stands for, for example `GET /api/groups/{id}`.
- The app uses **two APIs on purpose**, so you learn both: every screen **reads** its data with one **GraphQL** query, and every button that **changes** something calls **REST**. Log in and sign up are REST too.
- **Friends don't need the app to be in a group.** You add them by name and split with them at once. When they join, they pick their name and everything recorded for them becomes theirs.
- When your FastAPI backend is ready, only the files in `src/api/` change. The screens stay the same.

1 Start here: run the app

```
cd ~/Desktop/Hisaab/frontend
npm install        # first time only
npm run dev        # then open http://localhost:5173
```

On the log in screen, press **Try the demo account**. It logs you in as Divas Sharma (`divas@hisaab.demo` , password `demo1234`). Every other sample person uses the same password, for example `riya@hisaab.demo` .

Links worth opening

Address	What it shows
`/`	Your groups and your overall balance.
`/groups/1`	Room 204 (hostel). Your balance, expenses, balances, activity.
`/groups/3/settle`	Goa trip settle up: the fewest payments for 5 people.
`/join/join-demo`	A working invite link to "PG Indiranagar", a group you are not in yet.
`/join/expired-demo`	An invite link that has expired.
`/design`	Every colour, text style and component on one page, with a light/dark switch.

The sample data

Group	Kind	People	What it shows off
Room 204, Godavari	Hostel	Divas, Aman, Karan, Rohit	Small daily expenses, a split by exact amounts, one UPI payment. Rohit has no UPI ID.
Flat 3B, Koramangala	Flat	Riya (admin), Divas, Sneha	Rent split by shares (Riya has 2 shares for the big room), bills, groceries.
Goa trip, October	Trip	Arjun (admin), Divas, Aman, Riya, Priya, and Kabir (added by name, not joined)	Large amounts (over ₹1 lakh in total), a 6-person settle up, and a friend who is not on Hisaab yet.
Navratri dandiya night	Other	Divas, Aman, Sneha	A group where everyone is settled up.
PG Indiranagar	Flat	Meera, Sneha, plus "Divas" (added by your email) and "Tanvi" (added by name)	You are not in it yet. Home shows "Meera added you" with Join / Not me. `/join/join-demo` asks "Which one are you?".

The data is saved in your browser's localStorage under the key `hisaab.mockdb.v1` . It survives a page reload. **Account** → **Reset demo data** puts the sample data back.

Tech stack

Piece	Version	Used for
React	19.3	Building the screens out of components.
Vite	8.3	Dev server and production build.

react-router	7.18	Page addresses (routes) and navigation. Pinned to v7 on purpose.
Tailwind CSS	4.3	Styling with utility classes. Colours and materials are defined once in `src/index.css`.
graphql	16.15	Only for the mock: it runs real GraphQL queries in the browser against the schema. Removed in step 5. Loaded separately, so it is not in the main bundle.

No other packages. The icons are hand-drawn SVGs in `Icon.jsx`. The spring animations are a 40-line file, `lib/spring.js`.

---

2 The big picture

The frontend is built in layers. Each layer only talks to the layer below it. This is what lets the mock data be swapped for your backend later without touching the screens.

Pages	One file per screen in `src/pages/`. A page decides what to show and what happens when you tap. Example: `GroupPage.jsx`.
↓ uses	
Components	Reusable pieces in `src/components/`: buttons, cards, the bottom sheet, the top bar, money labels, avatars.
↓ pages load data with	
useApi hook	`useApi(() => api.getGroup(id), [id])` loads data, shows a loading state the first time, and reloads quietly after any change anywhere in the app.
↓ calls	
API layer	**Reads:** `gql(GROUP_SCREEN, {id})` sends one GraphQL query (`src/api/graphql.js`, queries in `queries.js`). **Writes:** `api.createExpense(…)`, `api.createSettlement(…)`… are REST calls (`src/api/*.js`). The screens never touch data any other way.
↓ today these read and write	
Mock server (today)	`src/mock/`. A JSON "database" in localStorage, plus the same checks a real server would do: logged in? member of this group? valid input?
↓ in step 5 they will call instead	
FastAPI + Postgres (later)	Your backend: REST routers under `/api/…` and one Strawberry GraphQL endpoint at `/api/graphql`. Both call the same service functions. Same JSON shapes as the mock.

Why build a mock server at all?

- The UI could be built and tested completely before any backend exists.

- The mock server behaves like your future API: it returns errors with a status code and a `detail` message, like FastAPI does. So the screens already handle errors properly.
- The mock functions are a living draft of the API contract. Section 8 lists them.
- It has a fake network delay of 140 to 380 ms, so loading states are visible and tested.

Pages and addresses (routes)

Address	Screen	Who can open it
`/login`, `/signup`	Log in, create account	Logged-out people
`/welcome`	Add your UPI ID (right after sign up)	Logged in
`/`	Home (phone) / Overview (laptop)	Logged in
`/groups/new`	New group	Logged in
`/groups/:groupId`	Group screen. `?tab=balances` or `?tab=activity` picks a tab.	Members
`/groups/:groupId/add`	Add expense	Members
`/groups/:groupId/expenses/:id`	Expense detail	Members
`/groups/:groupId/expenses/:id/edit`	Edit expense	Members
`/groups/:groupId/settle`	Settle up	Members
`/groups/:groupId/members`	Members, invite link, group settings	Members
`/join/:token`	Join a group from an invite link	Logged in
`/account`	Profile, UPI ID, theme, log out	Logged in
`/design`	Design system page	Anyone

No frontend address starts with `/api`. The backend will live under `/api`, so the two can never clash. This avoids the SnipShare problem where refreshing `/users/…` showed JSON.

3 Folder and file map

Every file in `frontend/src`, with one line on what it does.

Entry and routes	
`main.jsx`	Starts React. Wraps the app in router, theme, auth and toast providers.
`App.jsx`	The list of routes (addresses → screens).
`index.css`	Tailwind, colour tokens for light and dark, glass materials, avatar colours, motion.
`api/`: what screens may call	
`index.js`	Exports `api` (REST writes) and `gql` (GraphQL reads). Holds the list of all REST routes and logs each call in dev.
`graphql.js`	The GraphQL client `gql(query, variables)`. Today it runs the query on the mock; later it POSTs to `/api/graphql`.
`queries.js`	Every GraphQL query the screens use, one per screen.
`schema.graphql`	The GraphQL schema: the read half of the API contract.
`devlog.js`	Prints **REST** and **GraphQL** calls in the browser console (development only).
`client.js`	Shared server pieces: fake delay, errors, token, "are you a member", "are you an admin", activity log.
`events.js`	The "data changed" signal. Writes send it; screens listen and reload.
`auth.js`	Sign up, log in, get me, update me, log out.
`groups.js`	REST writes: create, rename, leave, remove member, set role.
`invites.js`	REST: get or make the invite link, reset it, join with it.
`expenses.js`	REST writes: create, update, delete (soft), restore. All validation lives here.

`hooks/`	
`useApi.js`	Load data for a screen; reload quietly after any write.
`useScrolled.js`	Has the page scrolled? Turns the top bar into glass.
`useTitle.js`	Sets the browser tab title.
`useNow.js`	Re-renders every 30 s so countdowns stay correct.
`useMediaQuery.js`	Is the screen wide? The sidebar only loads on laptops.
`context/`: app-wide state	
`AuthContext.jsx`	Who is logged in. `RequireAuth` and `GuestOnly` guard the routes.
`ThemeContext.jsx`	Auto / light / dark, saved in localStorage.
`ToastContext.jsx`	Short messages at the bottom, optionally with Undo.
`components/`	
`AppShell.jsx`	Logged-in layout. On laptops adds the groups sidebar.
`AuthFrame.jsx`	Centred frame for log in, sign up, join.
`Icon.jsx`	All icons.
`ActivityItem.jsx`	Turns an activity record into a sentence.
`GroupRow.jsx`	One group in a list, with your balance.
`InviteCard.jsx`	Invite link: share, copy, countdown, reset.
`components/ui/`: building blocks	

`settlements.js`	REST writes: record a payment, remove one.
`ledger.js`	Server-side balance helpers used by REST and the GraphQL resolvers.
`mock/`	
`seed.js`	The sample people, groups, expenses and payments. Dates are relative to today.
`db.js`	Loads and saves the fake database in localStorage. Keeps two tabs in sync.
`resolvers.js`	Mock GraphQL resolvers: answer each schema field from the fake database.
`lib/`: plain helpers, no React	
`money.js`	Paise ↔ rupees, ₹1,20,000 formatting, amount input formatting.
`split.js`	Equal / exact / shares splitting with the leftover-paisa rule.
`settle.js`	Net balances, fewest payments (greedy), who owes whom.
`categories.js`	The 9 categories, guessing one from the description, the 4 group kinds.
`dates.js`	"Today", "Yesterday", "Mon, 2 Nov", "3 h ago", "23 h 12 m" left.
`upi.js`	Builds the `upi://pay` link; checks UPI ID format; phone or laptop?
`people.js`	Shows your own name as "You".
`spring.js`	Spring animation, flick projection, rubber-band edges.
`feedback.js`	Copy to clipboard, share sheet, small vibration.
`cn.js`	Joins CSS class names.

`Button.jsx`	Buttons (5 styles, 3 sizes) and icon buttons.
`Sheet.jsx`	Draggable glass bottom sheet; confirm sheet.
`Page.jsx`	Page frame, glass top bar, glass bottom bar, error page.
`Card.jsx`	Card, list, row, section header.
`Money.jsx`	"you owe ₹340" / "you get back ₹1,240" labels.
`Avatar.jsx`	Initials avatar, same colour per person everywhere.
`Tile.jsx`	Coloured icon square for categories and kinds.
`Segmented.jsx`	Tabs with a sliding thumb (Expenses / Balances / Activity).
`Field.jsx`	Labelled input with hint or error underneath.
`Skeleton.jsx`, `EmptyState.jsx`, `Brand.jsx`, `Splash.jsx`	Loading shapes, friendly empty screens, logo.
`pages/`	
`LoginPage`, `SignupPage`, `WelcomePage`, `HomePage`, `NewGroupPage`, `group/GroupPage` (+ `ExpensesTab`, `BalancesTab`, `ActivityTab`, `PaymentSheet`), `ExpenseFormPage`, `ExpenseDetailPage`, `SettleUpPage`, `MembersPage`, `JoinPage`, `AccountPage`, `DesignPage`, `NotFoundPage`.	

4 How data moves

Reading: how a screen gets its data

```
const screen = useApi(() => gql(GROUP_SCREEN, { id: 1 }), [groupId]);
// screen.data    → { group: {...} }, or undefined while loading
// group.error    → an ApiError with .status and .detail, if it failed
// group.loading  → true on the first load
```

1. The screen opens. `useApi` sends one GraphQL query, `GroupScreen`, asking for exactly the fields this screen shows.
2. The screen shows grey loading shapes (skeletons) until the answer arrives.
3. The server checks the token, checks that you are a member, runs a resolver for each field, and returns only what was asked for.
4. If it fails, the screen shows a clear message and a way out. A 403 shows "You are not in this group".

Writing: what happens when you add an expense

1. You type in the form. The form keeps its own state (amount text, description, who paid, split type…).
2. As you type, the form previews each person's share using `lib/split.js`. Problems show at once on the main button, for example "₹400 left to assign".
3. You press **Add ₹1,200**. The form sends one body to `api.createExpense(groupId, body)`. The amount is in paise.
4. The API checks the token, checks you are a member, and validates everything (section 6).
5. It **computes the splits again** itself. A server never trusts numbers worked out by the browser.
6. It saves the expense and its splits together, and adds an activity record.
7. It calls `notifyChange()`. Every `useApi` on screen reloads quietly: the list, the balance, the sidebar.
8. The app goes back to the group and shows a message: "*Dinner*" *added · ₹300 each*, with an **Undo** button.

REST and GraphQL: who does what

> **The rule**
> Screens **read** with GraphQL. Buttons **write** with REST. Log in, sign up and "who am I" stay REST.

Screen	Reads (GraphQL query)	Writes (REST)
Home	`HomeScreen`: my_invites, groups, my_balance, recent_activity in one request	`POST /api/members/{member_id}/accept` or `…/decline`
Laptop sidebar	`SidebarGroups`	none
Group	`GroupScreen`; `GroupActivity` only when the tab opens	`DELETE /api/settlements/{id}`
Add / edit expense	`ExpenseForm`	`POST /api/groups/{id}/expenses`, `PATCH /api/expenses/{id}`

Expense detail	`ExpenseScreen`	`DELETE /api/expenses/{id}`, `POST /api/expenses/{id}/restore`
Settle up	`SettleScreen`	`POST /api/groups/{id}/settlements`, `DELETE /api/settlements/{id}`
New group	none	`POST /api/groups` (with the names typed in)
Members	`MembersScreen`	add person, invites, edit name, roles, remove, leave
Join	`InvitePreview`	`POST /api/invites/{token}/join`
Log in, sign up, account	none	`POST /api/auth/…`, `GET` / `PATCH /api/users/me`

What a GraphQL request looks like

```
POST /api/graphql
{ "query": "query GroupScreen($id: Int!) { group(id: $id) { name my_balance_paise members { name }
} }",
  "variables": { "id": 1 } }

200 OK
{ "data": { "group": { "name": "Room 204, Godavari",
                       "my_balance_paise": 7925,
                       "members": [{ "name": "Divas Sharma" }, { "name": "Aman Verma" }, …] } } }
```

There is one URL for every read. The query text says which fields to return, so the Home screen gets in one request what REST needed three requests for.

GraphQL words you will meet

Word	Meaning	In Hisaab
Schema	The list of types and fields the server offers.	`src/api/schema.graphql`
Query	A request for some fields. You write only what the screen needs.	`src/api/queries.js`
Variables	Values sent next to the query text, like `$id`.	`{ "id": 1 }`
Nested field	Following a relationship inside the same request.	expense → group → members
Fragment	A named set of fields reused in several queries.	`ActivityFields`, `ExpenseFields`
Directive	Changes how a field runs.	`@include(if: $editing)` in `ExpenseForm`
Resolver	The server function that produces one field's value.	`src/mock/resolvers.js` now; Strawberry later
errors list	GraphQL usually answers HTTP 200. Failures are in `"errors"`, with `extensions.code`.	`FORBIDDEN`, `NOT_FOUND`, …

The N+1 problem (you will see it happen)

Every expense has a payer. If the server loads each payer with its own SQL query, a list of 50 expenses costs 1 query for the list plus 50 queries for the payers. In development, every GraphQL line in the browser console shows "User lookups while answering: …". The backend fix is a **DataLoader**: it collects all user ids asked for during one request and loads them with a single `WHERE id IN (…)` query.

Three decisions already made

- **snake_case in GraphQL too** (Strawberry: `auto_camel_case=False`). Both APIs return the same names, so the screens never convert `amount_paise` ↔ `amountPaise`.
- **Ids are `Int`**, not GraphQL's `ID` type. `ID` sends ids as strings, and in JavaScript `"1" === 1` is false.
- **The join page uses a small `InviteGroup` type, never `Group`.** Otherwise a non-member could ask for `invite { group { expenses } }` and read a group they are not in. Every way into a `Group` must check membership.

Watching it work

Run `npm run dev`, open DevTools → Console and click around. Blue **REST** lines are writes. Pink **GraphQL** lines are reads; click one to see its variables, query text and user lookups. In development React runs each effect twice on purpose (StrictMode), so each read appears twice. The production build runs it once.

Logging in and staying logged in

- Log in returns an `access_token`. The app saves it in localStorage under `hisaab.token`.
- On every request the API reads the token to know who you are. In the mock the token is just `mock.<user id>`. Your backend will issue a signed JWT, as in SnipShare.
- When the app starts, `AuthContext` calls `GET /api/users/me`. If that fails with 401, the token is removed and you are sent to log in.
- `RequireAuth` wraps every logged-in page. If you are logged out it sends you to `/login` and remembers where you were going. After log in (or sign up + UPI step) you land back there. This is how an invite link still works for someone who has no account yet.

Errors

Every API error has a `status` and a `detail` sentence, the same shape FastAPI uses (`{"detail": "..."}`). Screens show `detail` directly, so write these sentences for users, for example "Settle up first. You can leave once your balance is zero."

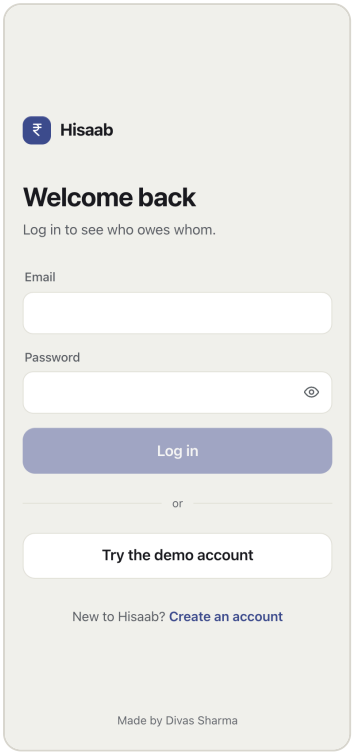
Status	Meaning in Hisaab	Example detail
401	Not logged in or token invalid	Please log in again.
403	Logged in, but not allowed	You are not a member of this group. / Only group admins can do this.
404	Does not exist (or was deleted)	This expense was deleted.
409	Conflicts with the current state	An account with this email already exists. / A group needs at least one admin.
410	Gone: the invite link expired	This invite link has expired.
422	Input is not valid	The split adds up to ₹1,100, but the expense is ₹1,200.

GraphQL errors carry the same meaning in `extensions.code`: `UNAUTHENTICATED` (401), `FORBIDDEN` (403), `NOT_FOUND` (404), `BAD_USER_INPUT` (422). The client turns them into the same `ApiError`, so screens handle REST and GraphQL errors the same way.

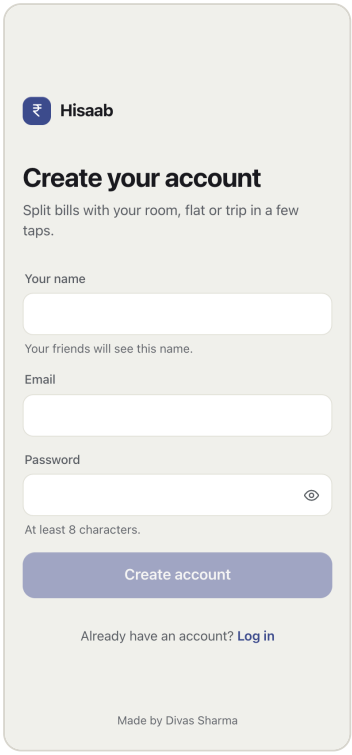
5 Screens, one by one

For each screen: what you see, what you can do, which API calls it makes, and why it is designed that way. Screenshots are from an iPhone-sized window (390 × 844).

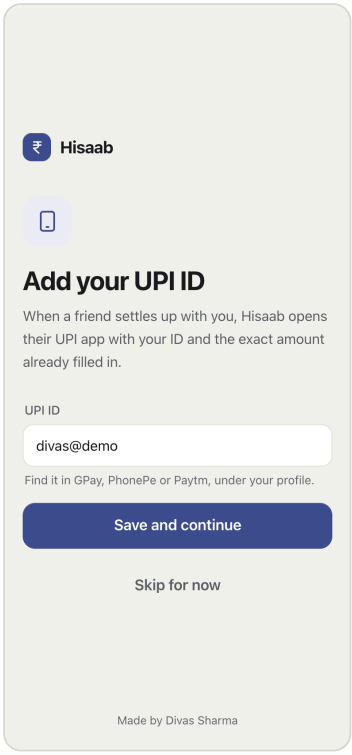
5.1 Log in, sign up, add UPI ID



Log in. "Try the demo account" logs in as Divas.



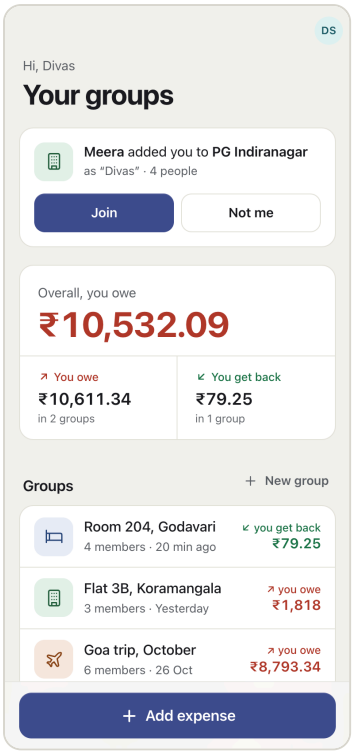
Sign up. The password hint counts down to 8 characters.



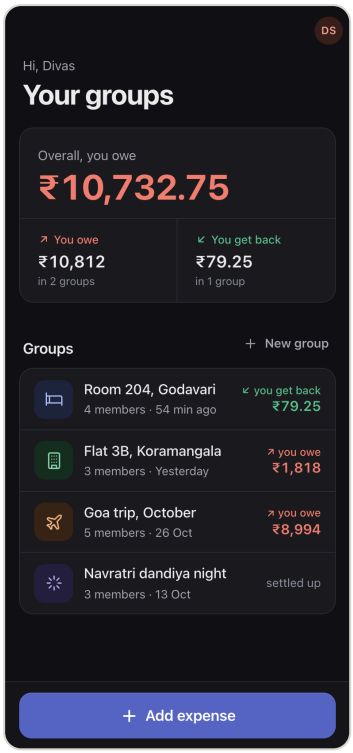
Right after sign up: add your UPI ID, or skip.

You can	Log in, create an account, show/hide the password, add a UPI ID or skip it.
API calls	`POST /api/auth/login`, `POST /api/auth/signup`, `PATCH /api/users/me`
Rules	Name 1–40 characters. Valid email, not already used (409). Password at least 8 characters. UPI ID looks like `name@bank`.
Why	The UPI ID is asked once, right after sign up, because settle up needs the payee's UPI ID. It is skippable so sign up stays fast.

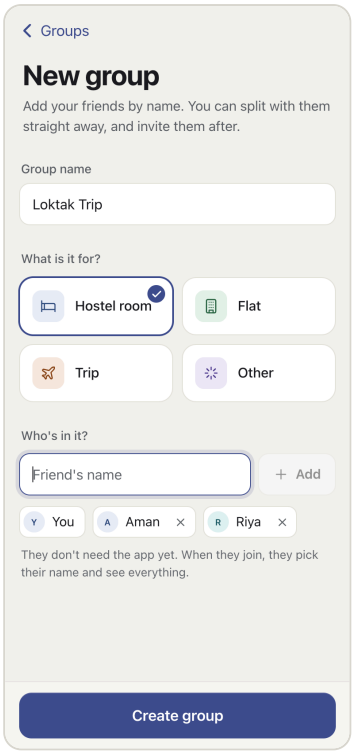
5.2 Home (phone) and Overview (laptop)



Home: an invite from someone who added you by email, your overall balance, then your groups.



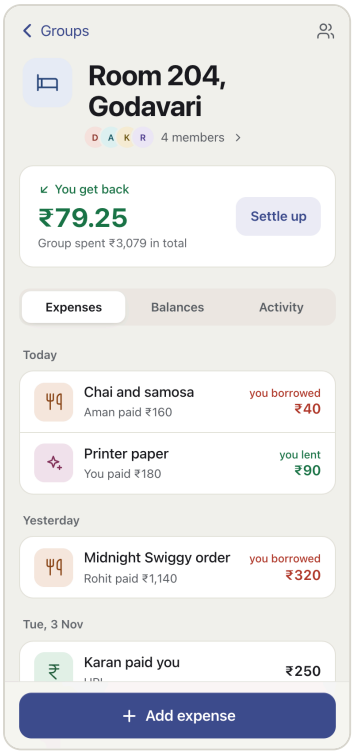
The same screen in dark mode.



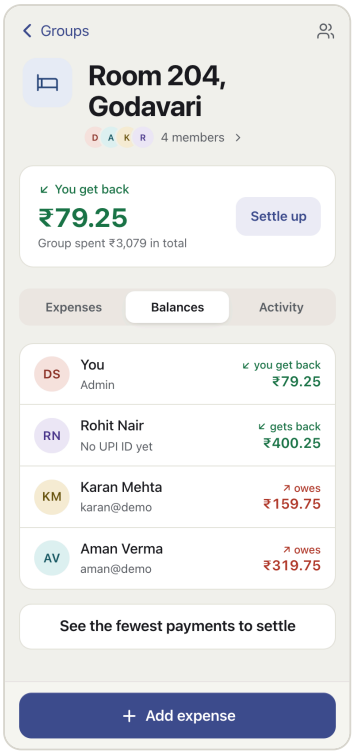
5.3 New group: name, kind, and "Who's in it?". Type a name, press Enter.

You see	"Overall, you owe ₹…" or "you get back ₹…", then two numbers: total you owe (and in how many groups) and total you get back. Then the group list, newest activity first.
Invites	If someone added you to a group with your email, a card says "Meera added you to PG Indiranagar, as "Divas"" with **Join** and **Not me**.
You can	Open a group. Create a group. Press **Add expense**: with one group it opens that group's form; with several, a sheet asks "Add an expense to…".
API calls	GraphQL `HomeScreen` (one request). The laptop sidebar uses GraphQL `SidebarGroups`. Join / Not me: REST `POST /api/members/{member_id}/accept` / `…/decline`.
New group	Name, kind, and the names of the people in it. You are added as admin. Everyone else is added by name ("not joined yet"). One request: `POST /api/groups`.
Empty state	With no groups: "No groups yet. Make one for your hostel room, your flat or your next trip." and a Create button.
Why	Adding an expense is the most common job, so it is the one big button in thumb reach. Each balance has a word and an arrow, not only a colour.

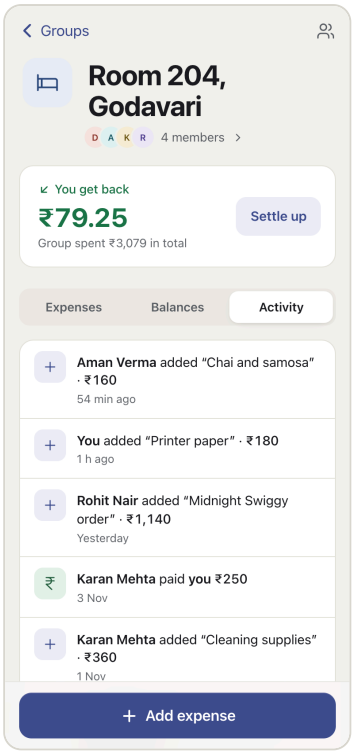
5.4 Group screen: Expenses, Balances, Activity



Expenses tab, grouped by day. Each row says what it means for you: "you lent" or "you borrowed".

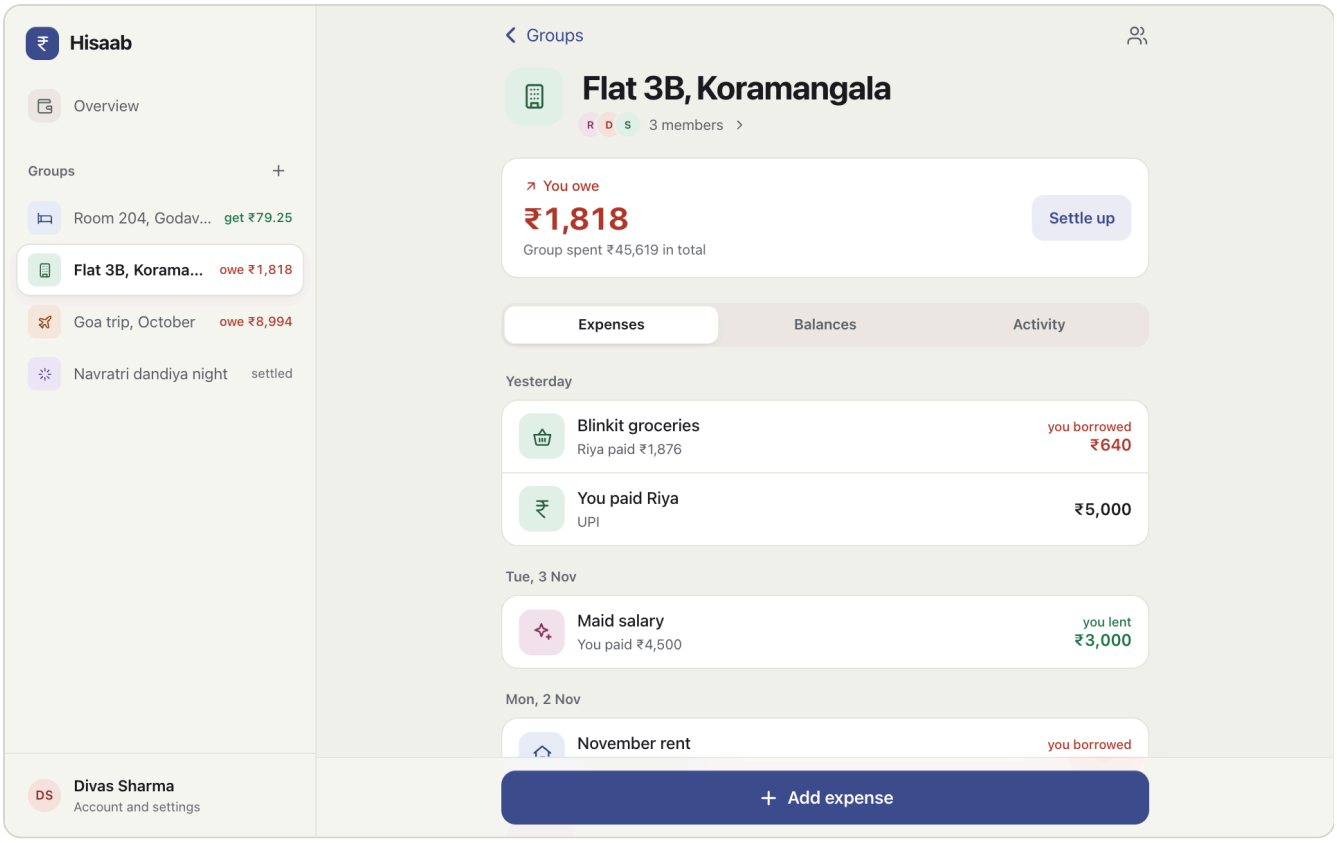


Balances tab: everyone's net balance, you first.



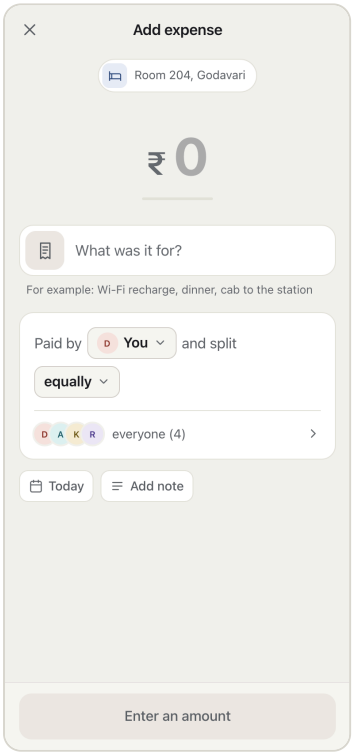
Activity tab: every change, as a sentence.

Top card	Your balance in this group ("You get back ₹79.25", "You owe ₹1,818" or "Settled up"), the group's total spending, and a **Settle up** button when anyone owes anything.
Expenses tab	Expenses and payments together, newest first, under day headings. Tap an expense to open it. Tap a payment to see it and remove it if it was a mistake (with Undo).
Balances tab	Each member's net balance: "gets back", "owes" or "settled up". Shows their UPI ID or "No UPI ID yet".
Activity tab	"Aman added "Chai and samosa" · ₹160", "Karan paid you ₹250", "Divas changed the amount of "Wi-Fi recharge"". 13 kinds of event (section 6.8).
Not joined yet	While friends added by name haven't joined, a card says "Aman and Riya haven't joined yet. You can already split with them." with an **Invite** button. If you are alone in the group, it shows the invite link and "Or add people by name".
API calls	GraphQL `GroupScreen`, and `GroupActivity` when the Activity tab opens. REST `DELETE /api/settlements/{id}` to remove a payment.

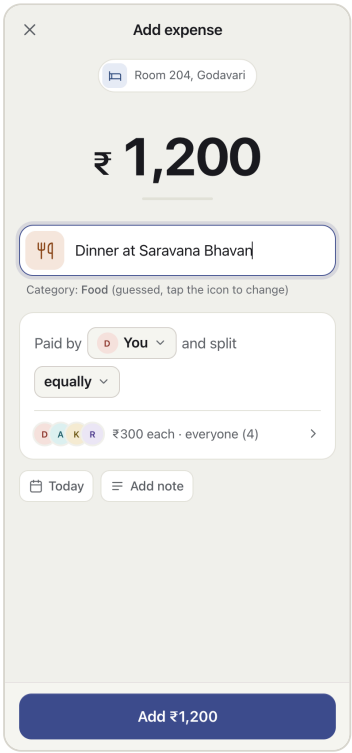


On a laptop (wider than 1024 px) the groups stay in a sidebar on the left. Each shows your balance in it.

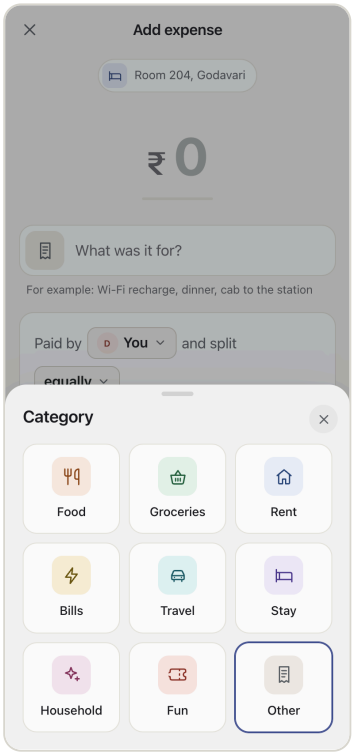
5.5 Add and edit an expense



Empty form. The number keypad opens at once. The bottom bar says what is missing.

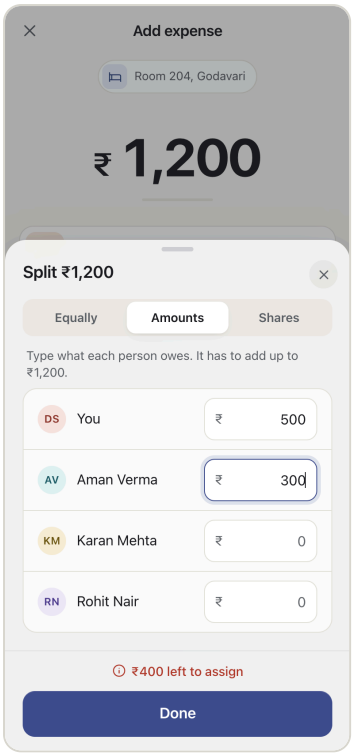


Filled in. Category guessed from "Dinner". The button shows the amount.

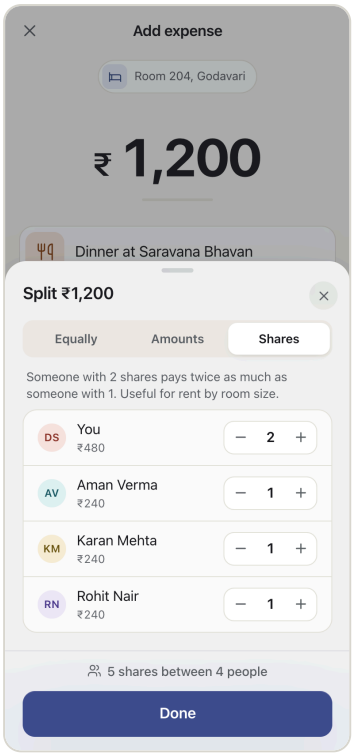


Tap the icon to pick a category yourself.

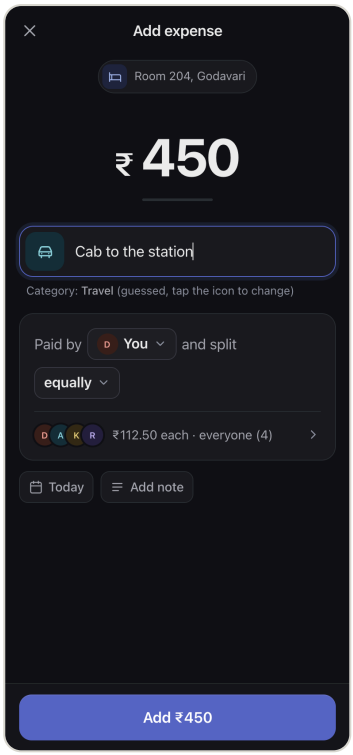
The form reads like a sentence: **Paid by [You] and split [equally]**. Each bracketed word is a button that opens a sheet. Under it, a summary line shows "₹300 each · everyone (4)".



Split by amounts. Live check: "₹400 left to assign".



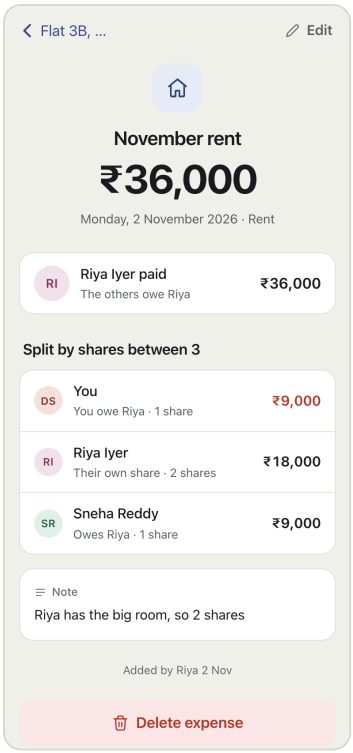
Split by shares, with – / + steppers and each person's amount.



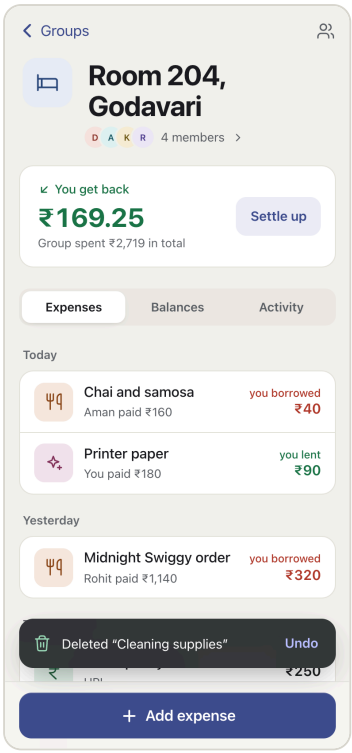
Dark mode. ₹450 split 4 ways is ₹112.50 each.

Fields	Amount (big, Indian commas as you type), description (max 60), category (guessed, can change), who paid, how to split, date (defaults to Today, can't be in the future), optional note (max 200).
Split: equally	Tick who shares it. Everyone ticked pays the same. Leftover paise go one each to the people with the lowest user ids (section 6.2).
Split: amounts	Type each person's amount. They must add up to the total. The sheet footer says "₹400 left to assign", "₹60 more than the total" or "Adds up to ₹1,200".
Split: shares	Give each person 0–99 shares. Someone with 2 shares pays twice as much as someone with 1. Good for rent by room size.
Main button	When something is missing it is replaced by a grey message that says exactly what: "Enter an amount", "Add a description", "Pick who to split with", "₹400 left to assign". When all is fine it reads **Add ₹1,200** (or **Save changes** when editing).
After saving	Goes back to the group. Message: "Pizza night" added · ₹250 each, with **Undo**.
API calls	GraphQL `ExpenseForm` (group members, plus the expense when editing). REST `POST /api/groups/{id}/expenses` or `PATCH /api/expenses/{id}` to save.
Why	The amount comes first because that is what people have in mind. The date is a visible chip because Splitwise hides it as a tiny icon and people miss it. Errors show while typing, not after pressing save.

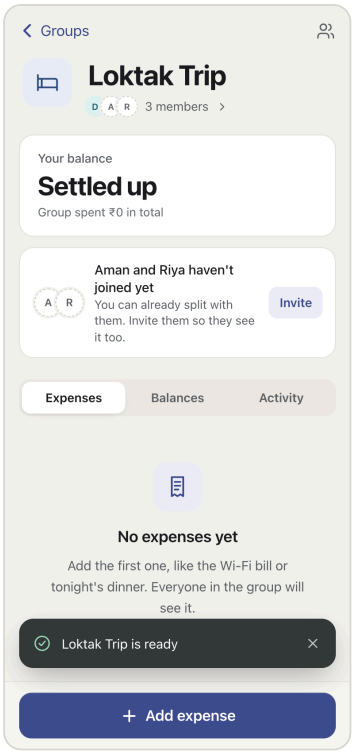
5.6 Expense detail



Rent split by shares. Your share is in red because you owe it.



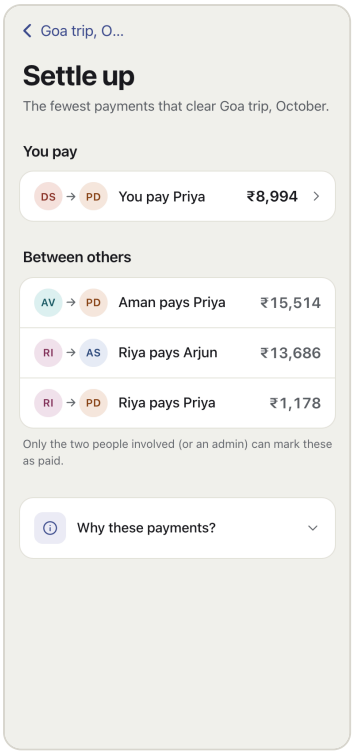
After Delete: no "Are you sure?". The message offers Undo instead.



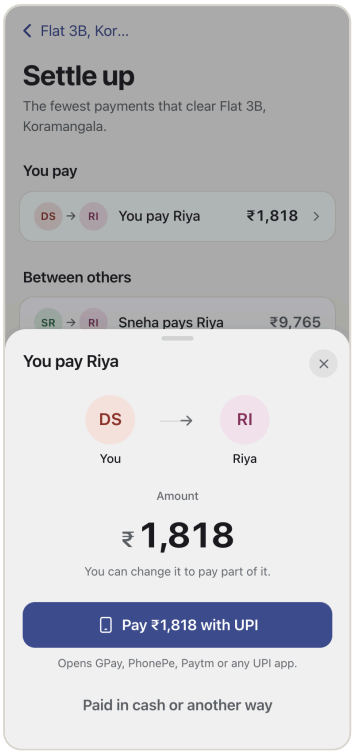
A new group made with two names: they haven't joined, and you can already split with them.

You see	Category, description, big amount, date. Who paid. Each person's share with "You owe Riya" or "Owes Riya", and shares if split by shares. The note. "Added by Riya 2 Nov · edited 1 h ago".
You can	Edit (top right). Delete (bottom).
Delete	Deletes at once and goes back to the group with an Undo message. Delete is a **soft delete**: the expense gets a `deleted_at` time and stops counting. Undo calls restore, which clears `deleted_at`.
API calls	GraphQL `ExpenseScreen` (the expense, and its group through a nested field). REST `DELETE /api/expenses/{id}`, `POST /api/expenses/{id}/restore`.
Safety	If the address has the wrong group id, the page redirects to the expense's real group.

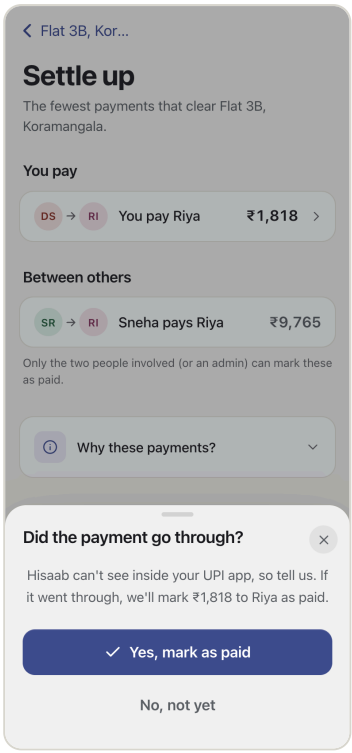
5.7 Settle up



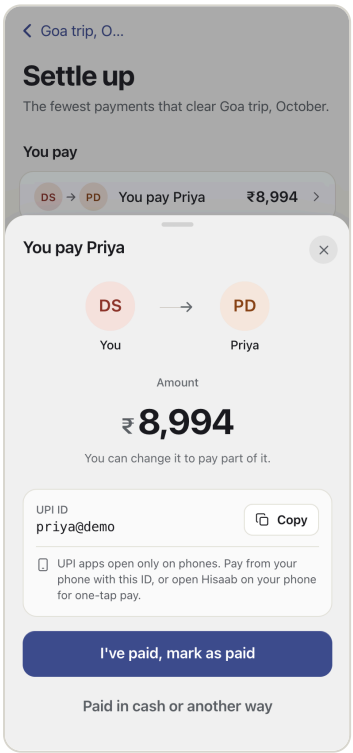
The fewest payments for the whole group, split into You pay / You get / Between others.



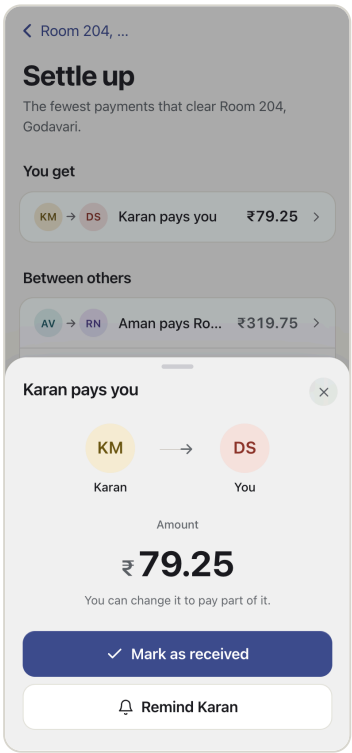
On a phone: one button opens any UPI app with the amount filled in.



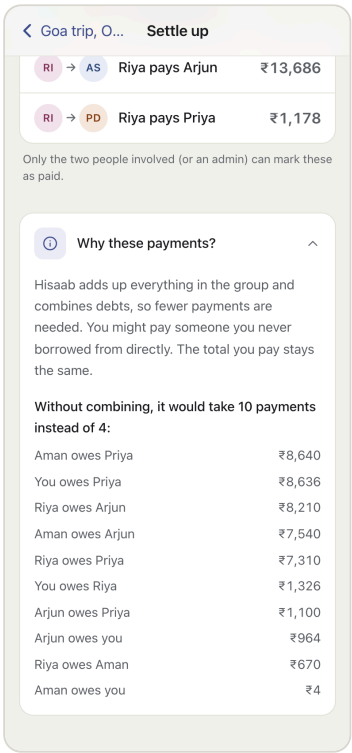
Back from the UPI app: "Did the payment go through?"



On a laptop UPI apps can't open, so the UPI ID is shown with Copy.



When someone owes you: Mark as received, or Remind them.



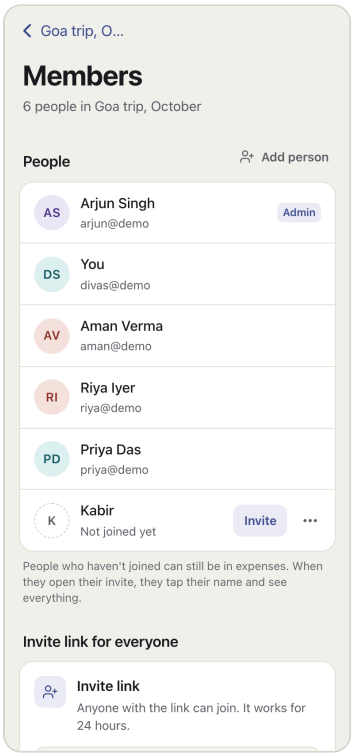
"Why these payments?" explains combining, and lists the uncombined debts.

The plan

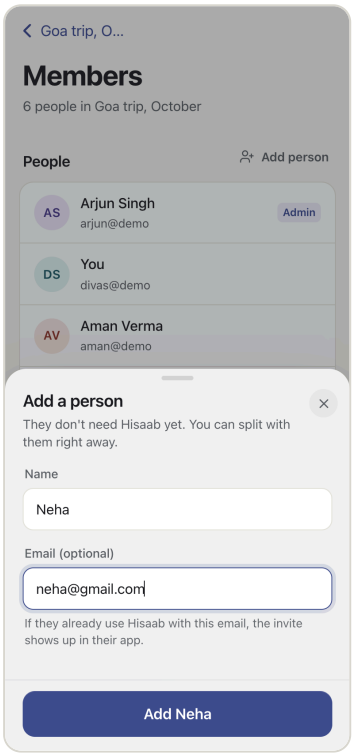
From the GraphQL `SettleScreen` query: `group.balances.suggested` is the fewest-payments list (section 6.4). Rows where you pay come first, then rows where you receive, then rows between other people.

You pay	The sheet shows You → Riya and the amount. You can lower it to pay part. On a phone: **Pay ₹1,818 with UPI** opens `upi://pay?pa=riya@demo&pn=Riya%20Iyer&am=1818.00&cu=INR&tn=Hisaab%20Flat%203B…`. A UPI link never reports back, so when you return to Hisaab it asks "Did the payment go through?". Only **Yes** records the payment. If the payee has no UPI ID, it says so and offers "Mark as paid".
You get	**Mark as received** records the payment. **Remind** opens the share sheet (or copies) with: "Hi Karan, you owe me ₹79.25 for Room 204, Godavari on Hisaab. My UPI ID is divas@demo."
Between others	Shown for information. Only an admin can mark these.
After paying	Message "Marked ₹8,994 to Priya as paid" with Undo. The plan recalculates. When nobody owes anything, the screen says "Everyone is settled up".
API calls	GraphQL `SettleScreen`. REST `POST /api/groups/{id}/settlements`, and `DELETE /api/settlements/{id}` for Undo.
Why	Tricount users often ask why they must pay someone they never borrowed from. The "Why these payments?" box answers that in two sentences.

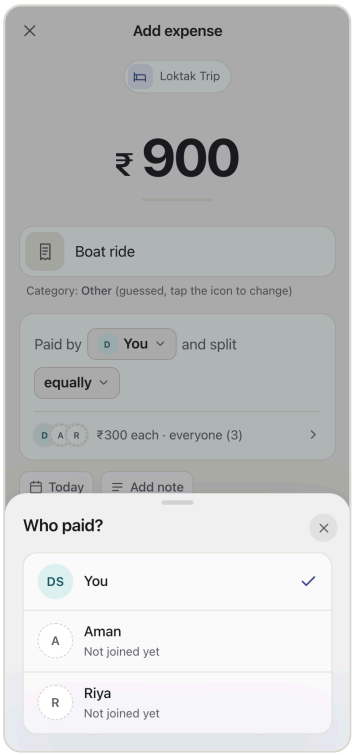
5.8 Members and group settings



People: joined members, and Kabir (dashed avatar) who hasn't joined. Invite is one tap.



Add a person: a name, and an email if you know it.

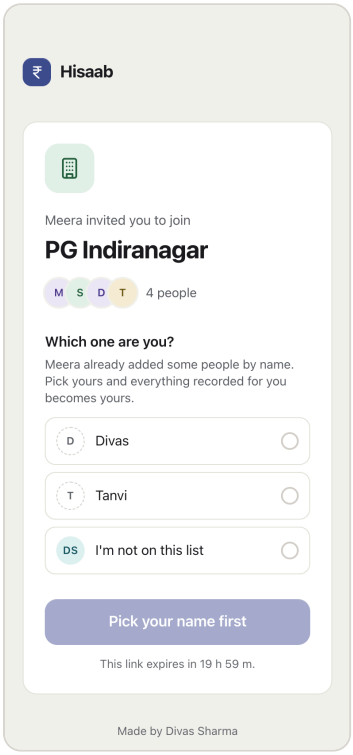


People who haven't joined can pay and be in splits like anyone else.

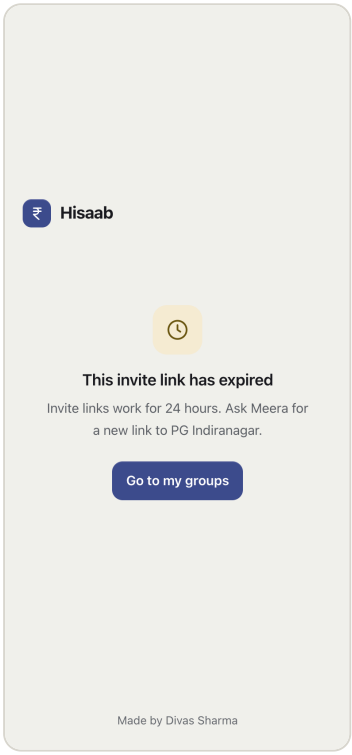
Add person	Any member can add someone by name, with an optional email. If that email belongs to a Hisaab account, the invite appears on their Home screen. Sending a real email needs the backend (a later ticket).
Invite one person	The **Invite** button next to someone who hasn't joined opens the share sheet (WhatsApp, SMS, email…) with: "Hi Aman, I added you to "Loktak Trip" on Hisaab… Open this link and tap your name to join: …/join/<token>?as=<member id>". The `?as=` part picks their name on the join page. On a laptop the message is copied.
Invite link	Any member sees the group's active link, when it expires ("Expires in 23 h 59 m"), and **Share** / **Copy link**. If no active link exists, opening the card creates one. Admins also see "Make a new link (the old one stops working)".

People	Joined people show their UPI ID (or "No UPI ID yet") and an Admin badge. People added by name show "Not joined yet". The ••• menu: for someone not joined, any member can Send invite or Edit name or email; admins can also Make admin / Remove admin rights (joined people only) and Remove from group.
Remove	Admins only, and only when that person's balance is zero. Asks for confirmation. It is a soft remove: their name still shows on old expenses.
Group details	Admins can rename the group and change its kind. Save appears only after a change.
Leave group	Blocked with a reason when your balance is not zero ("You still get back ₹79.25 here. Settle up first…"), or when you are the only admin and others remain ("Make someone else an admin first"). Otherwise asks for confirmation, leaves, and goes home.
API calls	GraphQL `MembersScreen`. REST `POST /api/groups/{id}/invites`, `POST /api/groups/{id}/invites/reset`, `PATCH /api/groups/{id}`, `POST /api/groups/{id}/members`, `PATCH /api/groups/{id}/members/{member_id}`, `DELETE /api/groups/{id}/members/{member_id}`, `DELETE /api/groups/{id}/members/me`

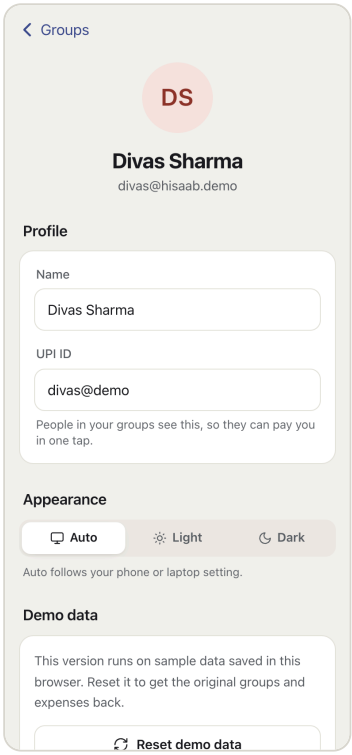
5.9 Join from an invite link, 5.10 Account



"Which one are you?": names added to the group that nobody has claimed.



Expired link: says what happened and who to ask.



Account: profile, UPI ID, theme, demo reset, log out.

Which one are you?	If the group has names nobody has claimed, the join page lists them, plus "I'm not on this list". Picking a name links your account to it, so its expenses and balance become yours. A personal link (`?as=13`) picks the name for you.
Join states	**valid** → Join button. **expired** → "This invite link has expired". **member** → "You're already in…" with Open. Unknown or reset link (404) → "This invite link doesn't work".
Join API	GraphQL `InvitePreview` (includes `open_members`), then REST `POST /api/invites/{token}/join` with `{"member_id": 13}`, or no body to join as a new person.
Account	Change name and UPI ID (`PATCH /api/users/me`). Theme: Auto (follows the device), Light, Dark. Reset demo data. Log out. Footer: "Hisaab · Made by Divas Sharma".

6 The rules: money, splits, balances

These rules live in `lib/` and `api/` today. Your backend must follow the same rules, or the numbers will not match.

Users and members (read this first)

A **user** is an account: someone who signed up. A **member** is a person inside one group. A member either has an account (`user_id` set, "joined") or is only a name someone typed (`user_id` empty, "not joined yet").

Expenses, splits and payments point to **members**, not users. That is what lets you split with Kabir before Kabir has the app. When Kabir joins and picks "Kabir", the member row gets his `user_id`. No expense has to change.

6.1 Money is stored as whole paise

- ₹1 = 100 paise. ₹1,200.50 is stored as `120050`.
- Never use floats for money. In floating point, 0.1 + 0.2 is not exactly 0.3. Integers are always exact.
- Every JSON field that holds money ends in `_paise`: `amount_paise`, `balance_paise`.
- Display uses Indian grouping: `₹1,20,000`. Whole rupees show no decimals; otherwise two decimals: `₹340.50`.
- The largest single expense allowed is ₹10,00,000 (`100000000` paise).

6.2 Splitting an expense, and the leftover-paisa rule

split_type	Input per person	How each share is worked out
`equal`	`{"member_id": 1}`	Everyone gets `floor(amount ÷ people)`.
`shares`	`{"member_id": 1, "shares": 2}`	Everyone gets `floor(amount × their shares ÷ total shares)`.
`exact`	`{"member_id": 1, "amount_paise": 50000}`	Exactly what was typed. Must add up to the amount, or 422.

Leftover-paisa rule (equal and shares)

Rounding down can leave a few paise unassigned. Give them out **one paisa each**, to the members in the split **sorted by member id, lowest first** (only people with shares > 0).

Example: ₹100 (10000 paise) split equally between members 3, 1 and 2. Each gets 3333. That is 9999, so 1 paisa is left. Member 1 (lowest id) gets it. Result: member 1 = ₹33.34, members 2 and 3 = ₹33.33. The total is exactly ₹100.

The browser computes splits only for the preview. The server always computes them again from the body and stores the result in `expense_splits`.

6.3 Net balance of one member in one group

```
net = (total of expenses this person paid)
    − (total of this person's shares)
    + (payments this person sent)
    − (payments this person received)

net > 0  → the group owes them    → shown as "gets back"
net < 0  → they owe the group     → shown as "owes"
net = 0  → "settled up"
```

Deleted expenses (`deleted_at` is set) do not count. In every group the nets add up to exactly 0. In SQL this is a few `SUM(...) GROUP BY member_id` queries.

The home screen's overall numbers add up your net across groups: "You owe" is the sum of your negative group balances, and "You get back" is the sum of your positive ones.

6.4 Fewest payments to settle (greedy)

1. Work out every member's net balance (6.3).
2. Put people with net > 0 in a "creditors" list and people with net < 0 in a "debtors" list.
3. Take the creditor who is owed the most and the debtor who owes the most. Ties: lower member id first.
4. The debtor pays the creditor `min(what one is owed, what the other owes)`. Add that payment to the plan.
5. Reduce both by that amount. Anyone who reaches 0 leaves their list.
6. Repeat until both lists are empty.

The frontend mock sorts the lists every round. Your backend version uses two heaps (Python's `heapq`) so each round takes the biggest values quickly. Same idea, same result.

Worked example

A pays ₹900 for dinner, split equally between A, B and C (₹300 each). B pays ₹300 for a cab, split equally between B and C (₹150 each).

Person	Paid	Shares	Net
A	₹900	₹300	**+₹600**
B	₹300	₹300 + ₹150	**−₹150**
C	₹0	₹300 + ₹150	**−₹450**

Without combining (who owes whom directly): B owes A ₹300, C owes A ₹300, C owes B ₹150. After netting B and A: B owes A ₹150. That is still 3 payments.

Greedy plan: C (owes most, ₹450) pays A (owed most, ₹600) ₹450. Then B pays A ₹150. **2 payments.** C never borrowed from A directly for the cab, but C's total is the same ₹450. This is what the "Why these payments?" box explains.

The API returns both lists: `suggested` (the greedy plan) and `pairwise` (who owes whom directly, after payments).

6.5 Invite links

- A link looks like `https://…/join/<token>`. It works for **24 hours** from when it was made.
- Each group has at most one active link. Asking for the link returns the active one, or makes a new one.
- An admin can reset it. All older links for that group stop working at once.
- **Personal links:** adding `?as=<member id>` makes the join page pick that name. The Members screen builds this link for each person who hasn't joined.
- **Claiming a name** must be one atomic update, so two people can't claim "Aman" at the same moment: `UPDATE members SET user_id = :me, joined_at = now() WHERE id = :member_id AND user_id IS NULL`. If 0 rows changed, someone got there first → 409.
- **In-app invites:** a member added with an email shows up in `my_invites` for the account with that email. **Join** claims the name; **Not me** clears the email so the invite disappears.
- **Email invites** (actually sending an email) need the backend: a later ticket.
- Preview statuses: `valid`, `expired`, `member` (you are already in). Unknown or reset token → 404. Joining with an expired token → 410.
- Your backend will sign the token (group id + expiry) so it can't be guessed or edited. To support "reset", either keep an invites table with a `revoked` flag, or store an invite version number on the group and put it inside the token. Pick one in step 2.

6.6 Soft delete and Undo

- **Expenses** are soft-deleted: `deleted_at` gets a time. Restore sets it back to null. Lists and balances ignore deleted expenses.
- **Payments** (settlements) are hard-deleted. Undo simply records the same payment again.
- Undo replaces "Are you sure?" everywhere it is safe. Only removing a member, leaving a group and resetting demo data ask for confirmation, because those can't be undone.

6.7 Validation rules (the server enforces all of these)

Thing	Rule
User name	1–40 characters after trimming spaces.
Email	Valid format, stored lower-case, unique.
Password	At least 8 characters. Store a bcrypt hash, never the password. (The mock stores it plainly; that is fine only for fake data.)
UPI ID	Empty, or matches `^[a-zA-Z0-9._-]{2,256}@[a-zA-Z][a-zA-Z0-9]{1,63}$`.
Group name	1–40 characters. Kind is one of `hostel`, `flat`, `trip`, `other`.
Description	1–60 characters.
Amount	Integer paise, greater than 0, at most 100000000.
Person added by name	Name 1–40 characters. Email optional, valid format, and nobody in the group may already have it (409).
Paid by	A member id of a current member (joined or not).
Category	One of `food`, `groceries`, `rent`, `bills`, `travel`, `stay`, `household`, `fun`, `other`.
Date	`YYYY-MM-DD`, not in the future.

Note	At most 200 characters.
Splits	At least one member. No member twice. Every `member_id` must be a current member of this group.
Exact splits	Each amount is an integer ≥ 0. The sum must equal the amount exactly.
Share splits	Each value is an integer ≥ 0. At least one is above 0.
Payment	Both member ids are current members and different. Amount is an integer above 0. Method is `upi` or `cash`.

6.8 Activity log

Every change adds one row: `{group_id, actor_id, type, data, created_at}`. `data` is a small JSON object. The 16 types:

type	data	Shown as
`group_created`	name	Divas created the group
`group_renamed`	from, to	Riya renamed the group to "…"
`member_added`	member_id, name	Arjun added Kabir
`member_joined`	member_id	Aman joined the group
`member_claimed`	member_id, as_name	Meera joined as "Kabir"
`member_renamed`	member_id, from, to	Divas renamed "Amn" to "Aman"
`member_left`	member_id	Sneha left the group
`member_removed`	member_id	Riya removed Sneha from the group
`role_changed`	member_id, role	Divas made Aman an admin
`expense_added`	expense_id, description, amount_paise	Aman added "Chai and samosa" · ₹160
`expense_edited`	expense_id, description, changes[]	Divas changed the amount of "Wi-Fi recharge"
`expense_deleted`	expense_id, description, amount_paise	Divas deleted "…" · ₹180
`expense_restored`	expense_id, description, amount_paise	Divas restored "…"
`payment_added`	settlement_id, from_member_id, to_member_id, amount_paise	Karan paid you ₹250
`payment_removed`	from_member_id, to_member_id, amount_paise	Divas removed a ₹250 payment
`invite_reset`	(none)	Divas made a new invite link

The activity API adds a name and an "is it me" flag for every member id in `data` (for example `from_member_id_name`, `from_member_id_is_me`), so the screen does not need extra requests. `changes[]` holds words from: description, amount, payer, category, date, note, split.

7 Who can do what

Every group request first checks that the caller is a member of that group. Then some actions also need the admin role. The person who creates a group becomes its admin.

Action	Allowed for	Extra condition
See a group, its expenses, balances, payments, activity	Members	Not a member → 403. Group doesn't exist → 404.
Add, edit, delete, restore an expense	Any member	Edits are logged in activity.
Record a payment	The payer, the receiver, or an admin	
Remove a payment	The payer, the receiver, whoever recorded it, or an admin	
Get and share the invite link	Any member	Creates a link if none is active.
Reset the invite link	Admins	Old links stop working.
Rename the group, change kind	Admins	
Make someone admin / remove admin	Admins	Only joined people can be admins. A group must keep at least one admin (409).
Add a person by name	Any member	Email optional; must not already be in the group.
Edit name or email of someone not joined	Any member	Joined people own their name (409).
Remove a member (joined or not)	Admins	Their balance must be 0 (409). Not yourself (use Leave).
Leave the group	You	Your balance must be 0. If you are the only admin and others remain, make someone else admin first (409).
Join with a link	Any logged-in person	Link must be active (410 if expired, 404 if unknown or reset). Joining twice does nothing.
Claim a name while joining	Anyone with a valid link	Only if nobody has claimed it (409). Done as one atomic update.
Accept or decline an in-app invite	Only the account whose email was typed	404 if the invite is gone or the email doesn't match.

The most important backend habit

Check membership on **every** request that takes a group id, an expense id or a settlement id. For an expense or settlement id, load the row first, then check membership of **its** group. Never trust a group id from the URL alone.

8 Draft API for step 2 (REST + GraphQL)

These are the calls the screens make today, so they are the starting draft for the contract. All money is in paise. All times are ISO 8601 strings. Everything needs `Authorization: Bearer <token>` except sign up and log in.

8.1 REST: auth and every write (21 endpoints)

Method and path	Body	Returns	Who
`POST /api/auth/signup`	name, email, password	`{access_token, user}`	Anyone
`POST /api/auth/login`	email, password	`{access_token, user}`	Anyone
`GET /api/users/me`		user	Logged in
`PATCH /api/users/me`	name?, upi_id?	user	Logged in
`POST /api/groups`	name, kind, members?: [{name, email?}]	`{id, name, kind}`	Logged in (becomes admin)
`PATCH /api/groups/{id}`	name, kind	`{id, name, kind}`	Admin
`DELETE /api/groups/{id}/members/me`		nothing (leave)	Member, balance 0
`POST /api/groups/{id}/members`	name, email?	member	Member
`PATCH /api/groups/{id}/members/{member_id}`	role, or name / email	member	Admin for role; any member for name/email of someone not joined
`DELETE /api/groups/{id}/members/{member_id}`		nothing (remove)	Admin
`POST /api/groups/{id}/invites`		`{token, expires_at}`	Member
`POST /api/groups/{id}/invites/reset`		`{token, expires_at}`	Admin
`POST /api/invites/{token}/join`	member_id? (the name you are)	`{group_id}`	Logged in
`POST /api/members/{member_id}/accept`		`{group_id}`	The account whose email was typed
`POST /api/members/{member_id}/decline`		nothing	The account whose email was typed
`POST /api/groups/{id}/expenses`	expense body	expense	Member

`PATCH /api/expenses/{id}`	expense body	expense	Member
`DELETE /api/expenses/{id}`		nothing (soft delete)	Member
`POST /api/expenses/{id}/restore`		expense	Member
`POST /api/groups/{id}/settlements`	from_member_id, to_member_id, amount_paise, method	payment	Payer, receiver or admin
`DELETE /api/settlements/{id}`		nothing	Payer, receiver, recorder or admin

8.2 GraphQL: every screen's reads (one endpoint)

`POST /api/graphql` with `{"query": "…", "variables": {…}}`. The full schema, with a comment on every field, is in `frontend/src/api/schema.graphql`. These are the entry points (root fields):

Query field	Returns	Check
`groups`	`[Group]`, newest activity first	Only my groups
`my_balance`	`OverallBalance`	Mine only
`recent_activity(limit)`	`[Activity]`	Only my groups
`group(id)`	`Group`	Member, or `FORBIDDEN`; exists, or `NOT_FOUND`
`expense(id)`	`Expense`	Member of its group; not deleted, or `NOT_FOUND`
`my_invites`	`[PendingInvite]`	Members added with my email, not yet answered
`invite(token)`	`InvitePreview`	Any logged-in person; `NOT_FOUND` if unknown or reset

Types and their fields

Type	Fields (bold = nested object, where a DataLoader helps)
`Group`	id, name, kind, created_at, my_member_id, my_role, my_balance_paise, total_spent_paise, member_count, last_activity_at, members, past_members, expenses(limit), settlements, balances, activity(limit)
`Member`	id (member id), user_id (null until joined), name, upi_id, joined, role, joined_at, invite_email, balance_paise
`User`	id, name, upi_id
`Expense`	id, group_id, description, amount_paise, paid_by, **payer**, split_type, category, date, note, created_by, **creator**, created_at, updated_at, splits, **group**
`Split`	member_id, **member**, amount_paise, shares
`Settlement`	id, group_id, from_member_id, to_member_id, **from_member**, **to_member**, amount_paise, method, created_by, created_at
`Transfer`	from_member_id, to_member_id, **from_member**, **to_member**, amount_paise

`Balances`	suggested, pairwise
`OverallBalance`	owed_to_me_paise, i_owe_paise, net_paise, groups_owed, groups_owe
`Activity`	id, type, created_at, **actor**, data (JSON), **group**
`InvitePreview`	status, expires_at, **invited_by**, group (an InviteGroup), open_members (names nobody has claimed)
`PendingInvite`	member_id, name, **added_by**, group (an InviteGroup)
`OpenMember`	id, name
`InviteGroup`	id, name, kind, member_count, members_preview

`paid_by`, `member_id`, `from_member_id` and `to_member_id` are member ids. `created_by` is a user id (the account that did it). Enums: `GroupKind` (hostel, flat, trip, other), `Role` (admin, member), `SplitType` (equal, exact, shares), `PaymentMethod` (upi, cash), `Category` (the 9 categories), `InviteStatus` (valid, expired, member).

JSON shapes (the same names in REST and GraphQL)

Expense body (REST create and edit)

```
{
  "description": "Dinner",
  "amount_paise": 120000,
  "paid_by": 1,
  "category": "food",
  "date": "2026-11-05",
  "note": "",
  "split_type": "equal",
  "splits": [{"member_id": 1}, {"member_id": 2}]
}
// exact:  [{"member_id": 1, "amount_paise":
70000}, ...]
// shares: [{"member_id": 1, "shares": 2}, ...]
```

Expense (returned)

```
{
  "id": 9, "group_id": 2,
  "description": "November rent",
  "amount_paise": 3600000, "paid_by": 5,
  "split_type": "shares", "category": "rent",
  "date": "2026-11-02", "note": "…",
  "created_by": 5,
  "created_at": "…", "updated_at": "…",
  "deleted_at": null,
  "splits": [
    {"member_id": 1, "amount_paise": 900000,
"shares": 1},
    {"member_id": 5, "amount_paise": 1800000,
"shares": 2},
    {"member_id": 6, "amount_paise": 900000,
"shares": 1}
  ]
}
```

A group in the GraphQL `groups` list

```
{ "id": 1, "name": "Room 204, Godavari",
  "kind": "hostel", "member_count": 4,
  "my_balance_paise": 7925,
  "last_activity_at": "…" }
```

A group from GraphQL `group(id)`

```
{ "id": 1, "name": "…", "kind": "hostel",
  "created_at": "…",
  "my_member_id": 1, "my_role": "admin",
  "my_balance_paise": 7925,
  "total_spent_paise": 307900,
  "members": [{
    "id": 1, "name": "Divas Sharma",
    "upi_id": "divas@demo",
    "joined": true, "role": "admin", "joined_at":
"…",
    "balance_paise": 7925 }],
  "past_members": [{"id": 9, "name": "…"}] }
```

Balances

```
{ "suggested": [{"from_member_id": 3,
                 "to_member_id": 1,
                 "amount_paise": 7925}],
  "pairwise":  [ same shape ] }
```

Overall balance (GraphQL `my_balance`)

```
{ "owed_to_me_paise": 7925, "i_owe_paise":
1081200,
  "net_paise": -1073275,
  "groups_owed": 1, "groups_owe": 2 }
```

Payment (settlement)

```
{ "id": 3, "group_id": 3,
  "from_member_id": 9, "to_member_id": 8,
  "amount_paise": 850000, "method": "upi",
  "created_by": 1, "created_at": "…" }
```

Invite preview (GraphQL `invite(token)`)

```
{ "status": "valid",
  "expires_at": "…",
  "invited_by": {"id": 9, "name": "Meera Joshi"},
  "group": { "id": 5, "name": "PG Indiranagar",
             "kind": "flat", "member_count": 2,
             "members_preview": [...] },
  "open_members": [{"id": 19, "name": "Divas"},
                   {"id": 20, "name": "Tanvi"}] }
```

`past_members` lets old expenses show names of people who have left the group.

---

9 Draft database tables for step 2

The mock database already has this shape. It maps directly to Postgres tables. Treat it as a first draft for you to change.

Table	Columns	Notes
users	id, name, email, password_hash, upi_id (nullable), created_at	email unique.
groups	id, name, kind, created_by → users, created_at	kind: hostel / flat / trip / other.
members	id, group_id → groups, user_id → users (**nullable**), name, invite_email (nullable), role, added_by → users, added_at, joined_at (nullable), removed_at (nullable)	Unique (group_id, user_id) where user_id is not null (a partial unique index). role: admin / member. Soft remove with removed_at.
expenses	id, group_id, description, amount_paise, paid_by → members, split_type, category, date, note, created_by, created_at, updated_at, deleted_at (nullable)	amount_paise is an integer. Soft delete with deleted_at.
expense_splits	expense_id → expenses, member_id → members, amount_paise, shares (nullable)	Primary key (expense_id, member_id). Save in the **same transaction** as the expense.
settlements	id, group_id, from_member_id → members, to_member_id → members, amount_paise, method, created_by → users, created_at	method: upi / cash.
invites (or a version on groups)	id, group_id, created_by, created_at, expires_at, revoked	Only needed if you store invites. See 6.5.
activity	id, group_id, actor_id → users, type, data (JSON), created_at	Append-only.

Where the project's learning goals show up

Backend skill (from CLAUDE.md)	Where the UI needs it
Many-to-many with roles	members (users ↔ groups, with a role); admin-only actions on the Members screen.

Nullable foreign key, partial unique index	`members.user_id` is empty until someone joins; one account can be in a group only once.
Race conditions	Claiming a name with one `UPDATE … WHERE user_id IS NULL`, and checking how many rows changed.
Transactions	Saving an expense and its splits together. If one insert fails, neither is saved.
Money as integer paise, leftover-paisa rule	Every amount; section 6.1–6.2.
SQL SUM / GROUP BY	Net balances (6.3), group totals, the overall balance on Home.
Permission check on every request	Section 7. The 403 screen already exists.
Signed, expiring invite tokens	Join page states: valid, expired, member, invalid.
Debt simplification (greedy + heaps)	`suggested` in the balances response; Settle up screen.
Fixing N+1 (selectinload, DataLoader)	GraphQL `groups` and `group.expenses` return many rows with nested payers and splits. Use selectinload for SQLAlchemy relationships and a DataLoader for GraphQL nested fields.
GraphQL (new)	A Strawberry schema matching `schema.graphql`; resolvers that call the same service functions as REST; a membership check in every way into a group; errors with `extensions.code`.
pytest against a test Postgres	The worked example in 6.4 and the ₹100 ÷ 3 example in 6.2 make good first tests. GraphQL is tested by POSTing a query to `/api/graphql`.

Suggested backend layout: one set of rules, two APIs

Folder in `backend/app/`	Files	Job
`services/`	`balances.py`, `expenses.py`, `groups.py`, `invites.py`	Every rule, written once.
`rest/`	`auth.py`, `groups.py`, `expenses.py`, `settlements.py`, `invites.py`	Thin FastAPI routers for auth and writes.
`graphql/`	`schema.py` (types + Query), `loaders.py` (DataLoaders), `context.py` (current user)	Strawberry schema for reads.
`main.py`		Includes the REST routers, and mounts `GraphQLRouter(schema)` at `/api/graphql`.

A REST route and a GraphQL resolver that need the same thing call the same service function. Then the money rules can never disagree between the two APIs.

10 Design system and UX reasons

Look

- **Page:** plain off-white (`#F4F3EF`). **Cards:** white with a thin border. No gradients, no colour blobs.
- **One accent:** muted indigo (`#3F4C8F`) for buttons and links. It is not green or red, because those two colours are kept for money.
- **Money colours:** green = money comes back to you, red = you owe. Always with a word ("you owe", "you get back") and an arrow, so colour is never the only signal.
- **Frosted glass** only on things that float over content: the top bar (once you scroll), the bottom bar, sheets and toasts. Bars: 72% opaque, 20 px blur. Sheets are a thicker material: 84%, 32 px blur. Devices set to "reduce transparency" get solid surfaces.
- **Type:** the system font (SF Pro on Apple devices, Roboto on Android). Tight letter spacing on big text, normal on body text. Digits in amounts are equal width (tabular) so columns of money line up.
- **Shapes:** rounded but not pill-shaped. Cards 16 px corners, main buttons 14 px, chips 10 px.
- **Dark mode:** every colour has a dark version. Auto follows the device; you can force Light or Dark in Account.

Motion

Interaction	Behaviour
Sheet opens	Spring, damping 1.0 (no bounce), response 0.38 s. Background dims as it rises.
Sheet dragged	Follows the finger 1:1 after 4 px. Dragging up past the top resists (rubber band).
Sheet released	Its speed is projected forward (Apple's formula, rate 0.998). If the projected position is past 45% of its height, it closes and carries the finger's speed into the closing spring. Otherwise it springs back with a slight bounce (damping 0.82).
Buttons	Shrink to 97% the moment they are pressed, not on release.
Tabs	The white thumb slides to the chosen tab.
Reduced motion	All slides and springs become instant; only quick fades remain.

UX psychology, applied

Principle	What it means	Where it is used
Hick's law	More choices = slower decisions.	One main button per screen. Split options stay hidden until you tap "equally".
Fitts's law / thumb zone	Big, close targets are faster to hit.	Main action is full width, 52 px tall, fixed at the bottom where the thumb rests.
Recognition over recall	Pick from what you see instead of remembering.	People as avatars with fixed colours; category guessed from the description; the "Paid by You and split equally" sentence.
Doherty threshold	Responses under ~400 ms keep people in flow.	Instant press feedback; skeletons while loading; quiet reloads keep old data on screen.

Forgiveness	Mistakes should be easy to reverse.	Undo for delete expense, mark paid, remove payment. Confirm only for things that can't be undone.
Feedback	Tell people what happened and what is wrong.	Live split checks; the main button says what is missing; toasts after every action.
Peak-end rule	People remember how an experience ends.	Settling up ends on a calm "Everyone is settled up" screen.
Jakob's law	People expect apps to work like apps they know.	"you lent / you borrowed" wording and the arrow diagram from Splitwise; Expenses / Balances tabs from Tricount.

Accessibility checks done

- Text contrast meets WCAG AA in both themes. Lowest pair in light mode: grey footnote on the page, 4.94 : 1. In dark mode: white on the accent button, 5.16 : 1.
- Visible focus rings; a "Skip to content" link; arrow keys move between tabs; Escape closes sheets; focus stays inside an open sheet and returns afterwards.
- Every icon-only button has a spoken label. Inputs are 16 px so iPhones do not zoom in.

---

11 Connecting the real backend (step 5)

1. Agree the final endpoints, JSON and schema in step 2. If anything changes, change the mock first (`src/api/` and `schema.graphql`), so the UI keeps working.
2. REST: add a small `apiFetch(path, options)` helper: base URL from `VITE_API_URL`, the Bearer token from localStorage, and FastAPI error parsing into `ApiError(status, detail)`. Replace the body of each REST function with a call to its endpoint.
3. GraphQL: in `src/api/graphql.js`, replace `mockTransport` with the `fetch` version already written there in a comment. The screens and `queries.js` do not change.
4. Keep calling `notifyChange()` after every REST write, so the GraphQL reads refresh.
5. Add a Vite dev proxy for `/api` to `http://127.0.0.1:8000`. It covers `/api/graphql` too.
6. Delete `src/mock/`, uninstall the `graphql` package, and remove the "Demo data" card in Account.

Example: one function before and after

```
// before (mock)
export function createExpense(groupId, body) {
  return request((s) => { /* validates and writes the mock database */ }, { write: true });
}

// after (real API)
export async function createExpense(groupId, body) {
  const res = await apiFetch(`/api/groups/${groupId}/expenses`, { method: 'POST', body:
JSON.stringify(body) });
  notifyChange();
  return res.json();
}
```

Checklist: what the backend must do that the UI relies on

- Return money as integer paise in fields ending with `_paise`.
- Compute splits on the server with the leftover-paisa rule; never trust client numbers.
- Match `schema.graphql` exactly: field names in snake_case (`auto_camel_case=False`), ids as `Int`, the same nullability.
- Mount Strawberry with `GraphQLRouter` at `/api/graphql`. Its `context_getter` returns the current user from the same JWT dependency the REST routes use.
- Send GraphQL errors with `extensions.code` set to `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND` or `BAD_USER_INPUT`.
- Point expenses, splits and payments at **members**; let a member exist without an account; claim names with one atomic update.
- Use soft delete for expenses and provide restore.
- Write an activity row for every change, with the data listed in 6.8.
- Send errors as `{"detail": "sentence for the user"}` with the status codes in section 4.
- Check membership and role on every request (section 7).

---

12 What was tested, and limits

Checked in a real browser (headless Chrome, phone and laptop sizes)

- Production build succeeds with no errors.
- Log in with the demo account; sign up → add UPI ID → land back on the invite link that started it.
- Add an expense with all three split types; live checks; save; the list and balances update.
- Delete an expense, then Undo: it comes back.
- Mark a payment as paid: the debt disappears from the plan. On a phone-sized browser the UPI button and the "Did the payment go through?" step appear.
- Join with the demo link; expired and unknown links show the right message.
- A group you are not in shows "You are not in this group". An expense opened under the wrong group redirects to the right one.
- Leave group is blocked for the only admin, and works after making someone else admin.
- Sheet drag: a short drag springs back, a long drag closes it.
- Dark mode and the laptop sidebar layout.
- Money maths: ₹100 ÷ 3, shares 2:1:1 on ₹36,000, net balances, the greedy plan.
- New group with two names; an expense paid by a friend who hasn't joined; paying them shows "hasn't joined Hisaab yet, so there is no UPI ID".
- A second account opens a personal link (`?as=13`), "Kabir" is already picked, joins as Kabir and takes over Kabir's ₹1,003.33 balance.
- The in-app invite on Home: Join opens the group with your share already there; Not me removes the card.
- Every screen's GraphQL query runs against the schema (an unknown field would fail as `GRAPHQL_VALIDATION_FAILED`). The console shows GraphQL for reads and REST for writes, as designed.
- A non-member asking for a group gets `FORBIDDEN`; an unknown invite gets `NOT_FOUND`.
- The laptop sidebar loads its data only on wide screens.

Limits of this version

- Data lives only in one browser. Two phones do not see each other's changes until the backend exists.
- The mock log in is not secure. Passwords are stored as plain text in the fake data, and the token is not signed.
- A UPI link can't confirm a payment. The user confirms it. Some UPI apps limit link payments to personal UPI IDs; nothing in the link can change that.
- The UPI IDs in the sample data end in `@demo`, which is not a real bank. Paying them will fail on purpose.
- Adding someone with an email does not send an email yet. That needs the backend.
- Not in v1: several people paying one expense, other currencies, receipt photos, recurring expenses, notifications, offline mode, search.
- The mock runs GraphQL inside the browser. There is no real network request until step 5.
- No automated frontend tests yet.

Next step

Step 2 in CLAUDE.md: agree the API contract. Sections 8 and 9 are the starting draft. After that comes step 3, the ticket board for the backend.